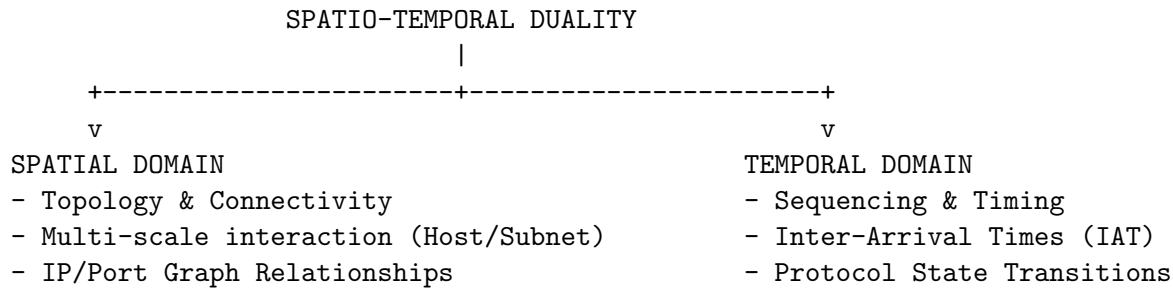


Part 1: Spatio-Temporal Telemetry Ingestion (The Senses)

1 The Nature of Network Data: Foundations & Duality

In cyber-physical and enterprise systems, network traffic is the continuous digital exhaust of distributed interactions. To train an AI World Model capable of causal state forecasting ($P(S_{t+1}|S_t)$), telemetry ingestion cannot treat network events as flat, independent rows in a tabular file. Network traffic possesses an inherent **spatio-temporal duality**:



1. **Spatial Domain:** The communication graph formed by IP addresses, subnets, broadcast domains, and destination ports. The network structure determines *reachability* and *lateral movement pathways*.
2. **Temporal Domain:** The exact sequencing, burstiness, inter-packet delays, and protocol flag progressions over time. Malicious campaigns are dynamic processes that unfold chronologically across attack stages.

1.1 The Dual-Resolution Dilemma: Flow vs. Packet Telemetry

Network telemetry traditionally presents an engineering trade-off between throughput and granularity:

The Architectural Solution: Project N-WORLD enforces a **Dual-Resolution Telemetry Engine**. We ingest raw packet streams or NetFlow data, extract micro-level temporal signatures (packet-level timing variations, flag patterns), and bind them into rolling-window macro-level state graphs.

2 Technical Taxonomy & Core Network Mechanics

To construct this telemetry layer, the ingestion pipeline must parse and structure specific Layer 3 (IPv4/IPv6) and Layer 4 (TCP/UDP/ICMP) protocol primitives.

Telemetry Type	Data Source / Standard	Strengths	Blind Spots / Failure Modes
Flow-Level (Macro)	NetFlow v9, IP-FIX, CICFlowMeter CSVs	High compression, low memory footprint, aggregate statistical overview	Collapses sequential ordering; completely misses slow-and-low evasive probes and fine-grained packet timing
Packet-Level (Micro)	Raw PCAP (Layer 2–Layer 4 frames)	Full protocol fidelity, exact microsecond timestamps, TCP flag state tracking	Massive storage overhead, computationally prohibitive for direct Transformer/GNN consumption at line rate

	0	1	2	3
	0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1			
IP	Version	IHL Type of Service	Total Length	
		Identification	Flags	Fragment Offset
		Time to Live	Protocol	
		Source Address		
		Destination Address		
TCP		Source Port		Destination Port
		Sequence Number		
		Acknowledgment Number		
	Data	U A P R S F		
	Offset	Reserved	R C S S Y I	Window
			G K H T N N	

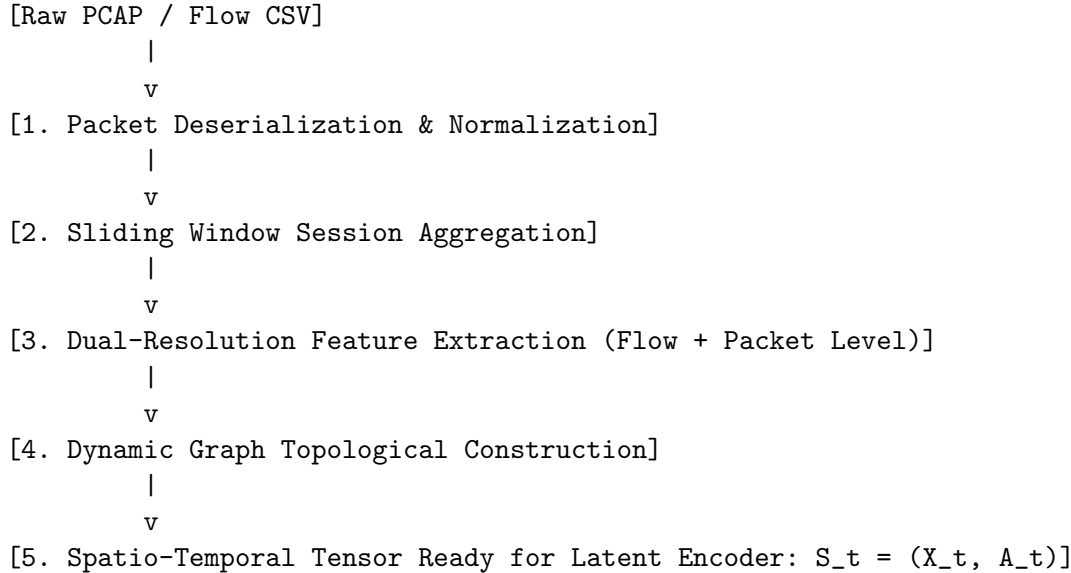
Key Technical Primitives

- **The 5-Tuple:** $\langle \text{Src IP}, \text{Dst IP}, \text{Src Port}, \text{Dst Port}, \text{Protocol} \rangle$. The atomic identifier defining a unique directional communication channel.
- **TCP Flag Bitmask (6-bit field):**
 - SYN (Synchronize): Establishes connections. Anomalous spikes without matching ACKs indicate SYN flood DoS or TCP half-open port scanning.

- ACK (Acknowledge): Confirms receipt. Used to measure bidirectional handshakes.
- PSH (Push) & URG (Urgent): Signals immediate data transmission. Frequently abused in out-of-band C2 signaling.
- RST (Reset) & FIN (Finish): Connection teardown. High RST/SYN ratios indicate port sweeping against closed services.
- **Time-to-Live (TTL) & TTL Variance:** Decrement by every Layer 3 hop. Variance within a single session suggests IP spoofing or dynamic routing shifts during data exfiltration.
- **TCP Window Size:** Dynamically advertised buffer capacity. Micro-fluctuations can expose low-rate Denial-of-Service attacks (e.g., Slowloris) and covert channel signaling.
- **Inter-Arrival Time (IAT):** The differential $\Delta t_i = t_i - t_{i-1}$ between successive packets within a flow. Periodic, low-variance Δt signatures are indicative of automated C2 beaconing.

3 Telemetry Pipeline: The What-How-Why Decomposition

Our ingestion pipeline transforms raw telemetry into a unified spatial tensor series $S_t = (\mathbf{X}_t, \mathbf{A}_t)$ over discrete time windows ΔT (e.g., $\Delta T = 1.0s$).



3.1 Step 1: Microsecond Packet Deserialization & Normalization

What

Ingesting raw binary packet captures (`.pcap`) or NetFlow records and parsing Layer 2–Layer 4 header fields into structured in-memory representations.

How

We employ a multi-threaded C-backed parsing worker pool using `PyShark` (wrapping `tshark`) and `Scapy`. Each packet is stripped of its raw Layer 7 payload (preserving privacy and compliance) while extracting raw header bytes, timestamps, and length distributions.

```

1 import pyshark
2 from scapy.all import rdpcap, IP, TCP, UDP
3 import numpy as np
4
5 def parse_packet_stream(pcap_file):
6     """
7     Extracts microsecond packet-level features for sequence modeling.
8     """
9     packets = rdpcap(pcap_file)
10    extracted_records = []
11
12    for pkt in packets:
13        if IP in pkt:
14            src_ip = pkt[IP].src
15            dst_ip = pkt[IP].dst
16            proto = pkt[IP].proto
17            ttl = pkt[IP].ttl
18            length = len(pkt)
19            timestamp = float(pkt.time)
20
21            # Layer 4 Extraction
22            flags = {"SYN": 0, "ACK": 0, "FIN": 0, "RST": 0, "PSH": 0, "URG": 0}
23            src_port, dst_port, win_size = 0, 0, 0
24
25            if TCP in pkt:
26                src_port = pkt[TCP].sport
27                dst_port = pkt[TCP].dport
28                win_size = pkt[TCP].window
29                tcp_flags = pkt[TCP].flags
30                flags["SYN"] = int(bool(tcp_flags & 0x02))
31                flags["ACK"] = int(bool(tcp_flags & 0x10))
32                flags["FIN"] = int(bool(tcp_flags & 0x01))
33                flags["RST"] = int(bool(tcp_flags & 0x04))
34                flags["PSH"] = int(bool(tcp_flags & 0x08))
35                flags["URG"] = int(bool(tcp_flags & 0x20))
36            elif UDP in pkt:
37                src_port = pkt[UDP].sport
38                dst_port = pkt[UDP].dport
39
40            extracted_records.append({
41                "timestamp": timestamp,
42                "src_ip": src_ip,
43                "dst_ip": dst_ip,
44                "src_port": src_port,
45                "dst_port": dst_port,
46                "proto": proto,
47                "ttl": ttl,
48                "win_size": win_size,
49                "length": length,
50                **flags
51            })
52    return extracted_records

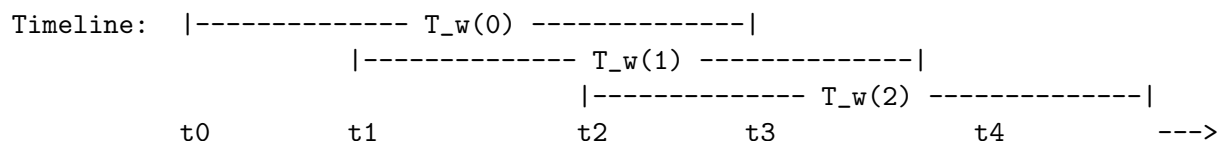
```

Why

Standard signature engines drop packet timings once flow tables are populated. Capturing precise arrival timestamps and per-packet TTL/Window sizes allows downstream models to catch evasive

3.2 Step 2: Temporal Windowing & Macro Flow Synthesis

Slicing continuous packet streams into discrete, overlapping temporal slices $T_w = [t_k, t_k + \Delta T]$ with stride δ , converting isolated packets into directional flow profiles.



Packets within window T_w are grouped by their 5-tuple identifier. For each flow f , we compute statistical aggregations:

$$\mu_{\text{IAT}} = \frac{1}{M-1} \sum_{i=2}^M (t_i - t_{i-1}), \quad \sigma_{\text{IAT}}^2 = \frac{1}{M-1} \sum_{i=2}^M ((t_i - t_{i-1}) - \mu_{\text{IAT}})^2 \quad (1)$$

$$\mathcal{R}_{\text{bytes}} = \frac{\text{Bytes}_{\text{forward}}}{\text{Bytes}_{\text{backward}} + \epsilon}, \quad \mathcal{R}_{\text{packets}} = \frac{\text{Packets}_{\text{forward}}}{\text{Packets}_{\text{backward}} + \epsilon} \quad (2)$$

```

1 import pandas as pd
2
3 def aggregate_temporal_window(records, window_duration=1.0):
4     """
5     Aggregates packet records into structured flow metrics over time window.
6     """
7     df = pd.DataFrame(records)
8     if df.empty:
9         return pd.DataFrame()
10
11     start_time = df['timestamp'].min()
12     df['window_idx'] = ((df['timestamp'] - start_time) // window_duration).astype(
13         int)
14
15     # Feature engineering per 5-tuple within window
16     grouped = df.groupby(['window_idx', 'src_ip', 'dst_ip', 'src_port', 'dst_port',
17         'proto'])
18
19     aggregated_flows = grouped.agg(
20         total_packets=('length', 'count'),
21         total_bytes=('length', 'sum'),
22         mean_ttl=('ttl', 'mean'),
23         std_ttl=('ttl', 'std'),
24         mean_win=('win_size', 'mean'),
25         syn_count=('SYN', 'sum'),
26         ack_count=('ACK', 'sum'),
27         rst_count=('RST', 'sum'),
28         psh_count=('PSH', 'sum'),

```

```

27     iat_mean=('timestamp', lambda x: np.mean(np.diff(x)) if len(x) > 1 else
0.0),
28     iat_variance=('timestamp', lambda x: np.var(np.diff(x)) if len(x) > 1 else
0.0)
29 ).reset_index()
30
31 # Handle NaN in variance for single-packet flows
32 aggregated_flows.fillna(0.0, inplace=True)
33 return aggregated_flows

```

Why

Sequence models require regular, discretized time indices. Collapsing asynchronous packet bursts into statistical window vectors bounds computational complexity while capturing the first and second moments of the traffic distribution.

3.3 Step 3: Graph Construction & Spatial Encoding

What

Translating the tabular flow records of window T_w into a topological graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ suitable for the Graph State Encoder.

How

- **Nodes ($\mathcal{V}_t$):** Unique endpoints (IPs or specific IP:Port pairs) active during T_w .
- **Node Feature Matrix ($\mathbf{X}_t \in \mathbb{R}^{|\mathcal{V}_t| \times d_{\text{feat}}}$):** Aggregated metrics associated with the node (e.g., outbound connection rate, total bytes sent/received, entropy of destination ports).
- **Edges ($\mathcal{E}_t$):** Active directional communication flows between endpoints.
- **Edge Feature Matrix ($\mathbf{E}_t \in \mathbb{R}^{|\mathcal{E}_t| \times d_{\text{edge}}}$):** Flow metrics (IAT variance, flag counts, byte volume) assigned directly to graph connections.

```

1 import torch
2
3 def build_graph_tensor(aggregated_flows):
4     """
5     Transforms windowed flow metrics into PyTorch-ready Graph representations.
6     """
7     unique_nodes = list(set(aggregated_flows['src_ip']).union(set(aggregated_flows
8 ['dst_ip'])))
9     node_to_idx = {ip: idx for idx, ip in enumerate(unique_nodes)}
10
11     num_nodes = len(unique_nodes)
12     edge_indices = []
13     edge_features = []
14
15     for _, row in aggregated_flows.iterrows():
16         u = node_to_idx[row['src_ip']]
17         v = node_to_idx[row['dst_ip']]
18         edge_indices.append([u, v])
19
20     # Feature vector for the edge

```

```

20     feat = [
21         row['total_packets'],
22         row['total_bytes'],
23         row['iat_mean'],
24         row['iat_variance'],
25         row['syn_count'],
26         row['ack_count'],
27         row['rst_count'],
28         row['mean_ttl']
29     ]
30     edge_features.append(feat)
31
32     edge_index = torch.tensor(edge_indices, dtype=torch.long).t().contiguous()
33     edge_attr = torch.tensor(edge_features, dtype=torch.float)
34
35     # Initialize basic node feature matrix (in-degree, out-degree, byte sums)
36     node_features = torch.zeros((num_nodes, 4), dtype=torch.float)
37     for _, row in aggregated_flows.iterrows():
38         u = node_to_idx[row['src_ip']]
39         v = node_to_idx[row['dst_ip']]
40         node_features[u, 0] += row['total_bytes'] # Outbound bytes
41         node_features[u, 1] += 1 # Out-degree
42         node_features[v, 2] += row['total_bytes'] # Inbound bytes
43         node_features[v, 3] += 1 # In-degree
44
45     return node_features, edge_index, edge_attr

```

Why

Traditional ML models treat an attack against an enterprise server the same as an attack against an isolated endpoint. Constructing a dynamic graph preserves the structural context of the enterprise—such as subnet boundaries and critical jump boxes—which is critical for forecasting lateral movement stages.

3.4 Step 4: Edge Case Mitigation & Fault-Tolerant Telemetry

INCOMING TELEMETRY ANOMALIES & STRATEGIC MITIGATIONS

-
1. Asynchronous Ingestion & Packet Drops
L-> Robust preprocessing via dynamic group-based statistical imputations
 2. Extreme Dimensionality / Port Sweeping (e.g., Nmap)
L-> Structural topology collapse to target IP with port-entropy node attributes
 3. Non-Stationary Drift & High-Throughput Jitter
L-> Robust Z-score normalization using rolling running median & IQR

Real-World Failure Modes & Solutions

1. Missing Data & Packet Drops in High-Throughput Streams:

- *Problem:* Saturated edge taps drop packets, causing broken time-series windows and missing statistical features.
- *Mitigation:* We apply a multi-tier imputation strategy. Rather than dropping the time slice or filling with arbitrary zeros (which biases the Transformer's attention weights), we use dynamic group-based imputation based on historical behavior of the same subnets.

2. Adversarial Flow Splitting & Out-of-Order Packets:

- *Problem:* Sophisticated attackers use IP fragmentation or split TCP payloads across out-of-order streams to evade flow reassembly.
- *Mitigation:* The packet parser maintains a short-lived micro-buffer that reconstructs overlapping fragments and normalizes TCP stream sequences before building edge feature vectors.

3. Graph Explosion during Mass Port Scans (e.g., SYN Floods):

- *Problem:* A scanner hitting 65,535 ports creates thousands of edges in a single second window, causing GNN out-of-memory (OOM) errors.
- *Mitigation:* We apply an edge collapse heuristic: when a single (Src_IP, Dst_IP) pair opens more than N connections within ΔT , we collapse them into a single aggregate edge and compute a high **Destination Port Entropy Score**:

$$H(\text{Ports}) = - \sum_{i=1}^K p(port_i) \log_2 p(port_i) \quad (3)$$

This captures the scanning behavior without overloading the graph representation.

4 End-to-End Pipeline Technical Stack

[Packet Capture Layer]	--> PyShark / Scapy / eBPF (C-backed Libpcap)
[Data Transformation]	--> Pandas / NumPy (Vectorized batch execution)
[Graph Tensor Generation]	--> PyTorch Geometric (Data / HeteroData structures)
[Offline Dataset Store]	--> CIC-IDS-2018 / CTU-13 (Standardized PCAP & CSVs)

The output of Part 1 is a clean, ordered sequence of graph objects:

$$\mathcal{S} = \{(\mathbf{X}_1, \mathbf{A}_1, \mathbf{E}_1), (\mathbf{X}_2, \mathbf{A}_2, \mathbf{E}_2), \dots, (\mathbf{X}_t, \mathbf{A}_t, \mathbf{E}_t)\} \quad (4)$$

This structured sequence feeds directly into **Part 2: Latent State Encoder (The Perception)**, where spatial message passing compresses each temporal graph slice into a dense vector z_t ready for our World Model's transition engine.

Part 2: Latent State Encoder (The Perception)

1 The Necessity of Non-Euclidean Representation

Traditional network intrusion detection methodologies often treat network telemetry identically to standard tabular data. A flow record is extracted, vectorized into an independent row, and fed through a static classifier (like a Random Forest or Multi-Layer Perceptron). This approach is fundamentally flawed for advanced threat forecasting because it completely discards the structural topology of the network.

In practice, network traffic inherently creates complex structures with nodes representing devices or users, and edges representing the communication channels. Advanced Persistent Threats (APTs) do not execute isolated actions; they move laterally, communicate with external Command-and-Control (C2) servers, and orchestrate coordinated multi-stage operations. When we evaluate an event in isolation, we lose the relational context. By modeling the entire network as a dynamic graph, we shift the perspective from isolated flows to system-level behavior, enabling the detection of anomalies that only manifest through the interaction of multiple entities.

The objective of the Latent State Encoder is to compress the spatio-temporal graph $\mathcal{G}_t$ (generated in Part 1) into a dense, continuous vector representation, z_t , while preserving these critical topological relationships.

2 Graph Formulation: Nodes, Edges, and Features

Before running our mathematical convolutions, we must establish how the network graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ is parameterized during a given temporal window T_w . Recent research has emphasized the significance of utilizing both node and edge features for multi-class intrusion detection, as traditional models often overlook critical edge data.

2.1 The Entities ($\mathcal{V}_t$ and $\mathcal{E}_t$)

- **Nodes ($\mathcal{V}_t$):** The vertices of our graph are the distinct communication endpoints. These can be represented as raw IP addresses or specific IP-Port combinations. Using port information is essential for identifying attacks, yet it is frequently omitted in standard training loops.
- **Edges ($\mathcal{E}_t$):** An edge connects two nodes if there is a detected data flow (unidirectional or bidirectional) during the time window. The edge contains flow-specific information such as duration, packet counts, and byte lengths.

2.2 The Feature Matrices

- **Node Attributes ($\mathbf{X}_t$):** For each node, we maintain a feature vector detailing its state (e.g., aggregate inbound/outbound volumes, connection degrees).
- **Edge Attributes ($\mathbf{E}_t$):** The dual-resolution metrics extracted in Part 1 (e.g., Inter-Arrival Time variance, TCP flag bitmasks, TTL distributions).

3 Spatial Encoding via Message Passing

To encode this graph into a meaningful latent feature space amenable to distinguishing normal behavior from anomalies, we deploy Graph Neural Networks (GNNs). GNNs combine information from nearby nodes, helping them learn both local and overall patterns in the network.

The core innovation of GNNs lies in the **message-passing mechanism**:

1. **Collection:** Each node collects information (messages) from its directly connected neighbors in the graph.
2. **Aggregation & Update:** It then updates its own representation (embedding) by combining its previous state with the aggregated neighborhood information using a learnable function.
3. **Propagation:** This process is repeated over multiple layers, expanding the “receptive field” of each node.

3.1 Graph Attention Networks (GAT)

For our specific World Model architecture, a standard Graph Convolutional Network (GCN) is insufficient because it treats all connected neighbors equally. In a cyberattack, a node might communicate with 50 benign internal servers and 1 malicious external IP.

We will utilize a **Graph Attention Network (GAT)** to dynamically calculate attention coefficients (α_{ij}) between connected nodes. This allows the model to learn the significance of specific connections, which is crucial for identifying coordinated attacks like Distributed Denial of Service (DDoS) originating from multiple sources or lateral movement propagating through connected hosts.

The mathematical update for a node i at layer $l + 1$ is defined as:

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij}^{(l)} \mathbf{W}^{(l)} h_j^{(l)} \right) \quad (1)$$

Where:

- h_i is the hidden state of node i .
- $\mathcal{N}(i)$ represents the immediate neighbors of node i .
- $\mathbf{W}$ is a learnable weight matrix.
- σ is a non-linear activation function (e.g., ELU).

3.2 Handling Heterogeneous Edge Features

Since edge features contain rich semantics for traffic behavior, we integrate edge attributes directly into the attention mechanism. The attention score between node i and j leverages the edge feature e_{ij} :

$$\alpha_{ij} = \text{softmax}_j \left(\text{LeakyReLU} \left(\mathbf{a}^T [\mathbf{W}h_i \parallel \mathbf{W}h_j \parallel \mathbf{W}_e e_{ij}] \right) \right) \quad (2)$$

4 Latent Compression (z_t) and Code Implementation

After L layers of message passing, each node has a highly contextualized embedding. However, our Transition Dynamics Engine (Part 3) requires a fixed-size vector to represent the *entire network state* at time t .

We apply a **Global Readout Operation** (e.g., Global Mean Pooling) across all updated node embeddings to generate the final latent state representation $z_t \in \mathbb{R}^d$.

Tech Stack: We use PyTorch and PyTorch Geometric to implement the spatial encoder.

```

1 import torch
2 import torch.nn.functional as F
3 from torch_geometric.nn import GATv2Conv, global_mean_pool
4
5 class LatentGraphEncoder(torch.nn.Module):
6     def __init__(self, node_dim, edge_dim, hidden_dim, latent_dim, heads=4):
7         super(LatentGraphEncoder, self).__init__()
8
9         # GATv2 allows for dynamic attention across both nodes and edge features
10        self.gat1 = GATv2Conv(in_channels=node_dim,
11                             out_channels=hidden_dim,
12                             heads=heads,
13                             edge_dim=edge_dim,
14                             concat=True)
15
16        self.gat2 = GATv2Conv(in_channels=hidden_dim * heads,
17                             out_channels=latent_dim,
18                             heads=1,
19                             edge_dim=edge_dim,
20                             concat=False)
21
22        def forward(self, x, edge_index, edge_attr, batch_index):
23            # x: Node features (N x node_dim)
24            # edge_index: Graph connectivity (2 x E)
25            # edge_attr: Edge features (E x edge_dim)
26
27            # Layer 1: Spatial Message Passing with Edge Attributes
28            x = self.gat1(x, edge_index, edge_attr)
29            x = F.elu(x)
30
31            # Layer 2: Deeper contextualization
32            x = self.gat2(x, edge_index, edge_attr)
33
34            # Global Pooling: Compress the entire graph into a single latent vector
35            z_t = global_mean_pool(x, batch_index)
36
37            return z_t

```

5 Architectural Edge Cases & The Pitch Strategy

When pitching this module to the hackathon judges, acknowledging the computational limitations of GNNs in real-world deployments demonstrates maturity.

Over-Smoothing and Graph Explosions

If a network is experiencing a massive DDoS attack, the target node’s degree will skyrocket. In standard GNNs, this creates a localized “black hole” during message passing, over-smoothing the embeddings and drowning out the subtle signals of concurrent, stealthy attacks. **Our Solution:** By heavily regularizing the multi-head attention weights and capping the maximum attention score a single node can receive, we isolate the topological bursts caused by volumetric attacks, ensuring the Latent State Vector z_t remains sensitive to all concurrent network behavior.

Categorical Feature Handling Challenges

Many heterogeneous synthetic datasets include categorical features corresponding to different network protocols (HTTP, TCP/IP, etc.). Standard numerical normalization fails here. **Our Solution:** We will pitch an *in-context pre-training* approach. By pre-training the GNN encoder on unlabeled network data using self-supervised learning (maximizing mutual information between local node patches and the global graph representation), the model naturally learns the semantic structure of categorical protocols before fine-tuning on labeled attack data.

The final output of this module is z_t , a dense multi-dimensional vector representing the topological and functional reality of the network at time t . This continuous stream of latent vectors, $Z = \{z_1, z_2, \dots, z_t\}$, is exactly what our World Model requires to forecast the future state of the attack timeline.

Part 3: Transition Dynamics Engine (The World Model)

1 The Paradigm Shift: From Classification to Simulation

If Part 1 (Telemetry) is the system's nervous system and Part 2 (Latent Encoder) is its visual cortex, then **Part 3: The Transition Dynamics Engine** is the prefrontal cortex. It is the engine of imagination.

To pitch this effectively, we must contrast it with traditional cyber defense. A standard Intrusion Detection System (IDS) is like a security guard looking at a static photograph of a bank lobby and trying to classify if a robbery is occurring. A **World Model** is akin to a chess grandmaster: it looks at the current board, understands the rules of the game (the network transition dynamics), and simulates the next K moves to realize, *“If this trajectory continues, checkmate (system compromise) occurs in 5 steps.”*

1.1 The Nature of the Data (The Latent Sequence)

The input to this engine is no longer raw network packets or flow statistics. The GNN from Part 2 has abstracted away the IP addresses, port numbers, and topological noise.

The data is now a continuous time-series of dense, continuous latent vectors:

$$Z_{\leq t} = \{z_1, z_2, z_3, \dots, z_t\}$$

Where each $z_i \in \mathbb{R}^d$ encapsulates the complete structural and behavioral state of the network at time window i . The mathematical objective of this engine is to learn the conditional probability distribution of the *next* state:

$$P(z_{t+1} | z_1, z_2, \dots, z_t)$$

2 Core Algorithm: The Autoregressive Temporal Transformer

While Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were historically used for time-series forecasting, they suffer from sequential bottlenecking and struggle to maintain context over long attack chains (e.g., an APT lying dormant for days).

For Project N-WORLD, we will pitch a **Bespoke Temporal Transformer**. Transformers leverage Self-Attention, allowing the model to directly correlate the current network state z_t with a reconnaissance scan that occurred far in the past (z_{t-500}), bypassing the fading memory problem of LSTMs.

3 The What-How-Why Decomposition

3.1 Step 1: Positional Encoding (Injecting Time)

- **What:** Adding chronological context to the latent vectors.
- **How:** Because Transformers process all inputs in parallel rather than sequentially, they have no inherent concept of order. We must inject a Positional Encoding vector p_t into each latent state z_t . We will use learnable positional embeddings or classic sine/cosine frequency functions:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (1)$$

- **Why:** Without positional encodings, an attack sequence of [Scan $\rightarrow$ Exploit $\rightarrow$ Exfiltrate] would look identical to the Transformer as [Exfiltrate $\rightarrow$ Scan $\rightarrow$ Exploit]. Time order is non-negotiable for causal simulation.

3.2 Step 2: Causal Masked Self-Attention

- **What:** Preventing the model from “cheating” by looking into the future during training.
- **How:** We apply a strict causal mask to the self-attention mechanism. The attention score between queries (Q) and keys (K) is masked using an upper-triangular matrix filled with $-\infty$.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (2)$$

Where $M_{i,j} = 0$ if $i \geq j$, and $-\infty$ otherwise.

- **Why:** During training, we feed the entire sequence $\{z_1, \dots, z_T\}$ into the model at once for efficiency. If we do not mask the future, the prediction for z_{t+1} will improperly attend to the actual z_{t+1} provided in the sequence, collapsing the model’s predictive capability during live inference.

3.3 Step 3: Latent State Prediction & Optimization

- **What:** Mapping the contextualized output back into the latent space format.
- **How:** The final layer of the Transformer outputs a hidden state h_t . We pass this through a Multi-Layer Perceptron (MLP) projection head to predict the exact numerical values of $\hat{z}_{t+1}$. The network is trained using supervised dynamics learning, minimizing the Mean Squared Error (MSE):

$$\mathcal{L}_{dynamics} = \frac{1}{T} \sum_{t=1}^T \|z_t - \hat{z}_t\|_2^2 \quad (3)$$

- **Why:** By forcing the model to continuously predict the next topological state of the network, it internalizes the “physics” of normal network operations and the specific phase-transitions of cyberattacks.

4 The K-Step Forward Simulation (Rollout)

The true power of the World Model is unlocked during inference via **Autoregressive Rollout**.

When a live network state sequence $Z_{\leq t}$ is ingested, the engine predicts the immediate future state $\hat{z}_{t+1}$. Rather than stopping there, the engine *appends* its own prediction to the input sequence, creating a synthetic history: $\{z_1, \dots, z_t, \hat{z}_{t+1}\}$. It then feeds this synthetic history back into itself to predict $\hat{z}_{t+2}$.

This loop repeats for K steps, generating a forecasted trajectory of the network: $\hat{Z}_{future} = \{\hat{z}_{t+1}, \hat{z}_{t+2}, \dots, \hat{z}_{t+K}\}$.

The Pitch Edge Case (Error Accumulation): A common critique of autoregressive models is that small errors in $\hat{z}_{t+1}$ compound recursively, causing $\hat{z}_{t+10}$ to drift into nonsense. To address this for the judges, we will specify that our training regime utilizes **Scheduled Sampling** (decaying teacher forcing). During training, we randomly replace the ground-truth z_t with the model's own previous prediction $\hat{z}_t$, forcing the Transformer to learn how to recover from its own minor simulation errors.

5 Technical Stack & Implementation

Tech Stack: Pure PyTorch. We will define custom classes to ensure we maintain absolute control over the causal masking and the autoregressive loop, which is critical for edge-deployment optimizations later.

```

1 import torch
2 import torch.nn as nn
3 import math
4
5 class CausalTemporalTransformer(nn.Module):
6     def __init__(self, latent_dim, num_heads, num_layers, hidden_dim):
7         super().__init__()
8         self.latent_dim = latent_dim
9
10        # Positional Encoding (Learnable for sequence flexibility)
11        self.pos_encoder = nn.Parameter(torch.zeros(1, 5000, latent_dim))
12
13        # Transformer Encoder configured for Causal generation
14        encoder_layers = nn.TransformerEncoderLayer(
15            d_model=latent_dim,
16            nhead=num_heads,
17            dim_feedforward=hidden_dim,
18            batch_first=True
19        )
20        self.transformer_encoder = nn.TransformerEncoder(encoder_layers,
21            num_layers)
22
23        # Projection head to output the exact latent dimensions
24        self.state_predictor = nn.Linear(latent_dim, latent_dim)
25
26    def generate_square_subsequent_mask(self, sz):
27        """Generates an upper-triangular matrix of -inf, with zeros on diag."""
28        mask = (torch.triu(torch.ones(sz, sz)) == 1).transpose(0, 1)
29        mask = mask.float().masked_fill(mask == 0, float('-inf')).masked_fill(mask
30        == 1, float(0.0))
31        return mask

```

```

31 def forward(self, src):
32     # src shape: [batch_size, seq_len, latent_dim]
33     seq_len = src.size(1)
34
35     # 1. Add Positional Encoding
36     src = src + self.pos_encoder[:, :seq_len, :]
37
38     # 2. Generate Causal Mask to prevent future-peeking
39     mask = self.generate_square_subsequent_mask(seq_len).to(src.device)
40
41     # 3. Pass through Transformer
42     output = self.transformer_encoder(src, mask=mask)
43
44     # 4. Predict next state
45     next_state_pred = self.state_predictor(output)
46     return next_state_pred
47
48 def rollout(self, current_sequence, k_steps):
49     """Autoregressive simulation of K future states."""
50     self.eval()
51     simulated_trajectory = []
52     seq = current_sequence.clone()
53
54     with torch.no_grad():
55         for _ in range(k_steps):
56             # Predict the next step
57             predictions = self.forward(seq)
58             next_state = predictions[:, -1:, :] # Take the last predicted
timestep
59
60             simulated_trajectory.append(next_state)
61
62             # Append prediction to the sequence, drop the oldest state to
maintain window
63             seq = torch.cat([seq[:, 1:, :], next_state], dim=1)
64
65     return torch.cat(simulated_trajectory, dim=1) # [batch_size, k_steps,
latent_dim]

```

This sequence of simulated latent vectors ($\hat{Z}_{future}$) represents our causal forecast. In the final phase (Part 4), we will feed this trajectory into the **Oracle**—our explainable forecasting head—to translate these numerical embeddings into actionable MITRE ATT&CK alerts.

The Oracle and the SOC “Glasshouse” Portal

1 The Oracle’s Input: Translating the Latent Future

The Transition Engine (Part 3) outputs a multidimensional trajectory tensor:

$$\hat{Z}_{\text{future}} = \{\hat{z}_{t+1}, \hat{z}_{t+2}, \dots, \hat{z}_{t+K}\}.$$

This tensor consists of pure numerical data arrays representing the forecasted structural and behavioral states of the network over the next K time windows. The Oracle’s function is to decode this abstracted, high-dimensional simulation back into actionable, human-readable cyber defense intelligence, answering three questions: *Will we be compromised? How? And why?*

2 Dual-Engine Classification: CatBoost & Semantic FAISS

To translate the latent trajectory into standard cybersecurity frameworks, we deploy a hybrid inference pipeline optimized for speed and accuracy on the edge.

- **What:** We map the simulated numerical latent vectors to a continuous infiltration probability $[0, 1]$ and a discrete MITRE ATT&CK stage (e.g., Lateral Movement, Exfiltration).
- **How:** We optimize a binary classification and multi-class pipeline using **CatBoost**. Its exceptional mastery over purely numerical arrays allows it to define robust, non-linear decision boundaries within the latent space without overfitting. Concurrently, we utilize a local vector database (FAISS) to compute the L_2 distance between the forecasted state $\hat{z}_{t+k}$ and a database of known, historical anomalous vectors.
- **Why:** Relying solely on a static classifier risks missing zero-day topological shifts. Combining a structured tree-based ensemble with semantic similarity matching creates a highly resilient forecasting head that mathematically proves its reasoning to the judges.

```
1 import catboost
2 import shap
3
4 class ExplainableOracle:
5     def __init__(self, model_path):
6         self.clf = catboost.CatBoostClassifier().load_model(model_path)
7         self.explainer = shap.TreeExplainer(self.clf)
8
9     def decode_trajectory(self, z_future):
10         # z_future: purely numerical simulated tensor
11         risk_probs = self.clf.predict_proba(z_future)[: , 1]
12         mitre_stages = self.clf.predict(z_future)
13         attributions = self.explainer.shap_values(z_future)
14         return risk_probs, mitre_stages, attributions
```

3 Explainability (XAI): Temporal & Spatial Attribution

A black-box prediction is a critical liability in enterprise environments. The Oracle must systematically unpack its logic to provide defenders with immediate root-cause analysis.

- **Temporal Attribution (The ‘When’):** We extract the attention weights (α) from the upstream Temporal Transformer. If the Oracle flags an infiltration at $t + 5$, the attention matrix explicitly reveals which historical data window (e.g., a subtle port scan at $t - 12$) catalyzed the trajectory.
- **Spatial & Feature Attribution (The ‘Where’ & ‘What’):** We execute SHapley Additive exPlanations (SHAP) directly on the CatBoost classifier. This isolates the exact underlying telemetry—such as anomalous TCP flag distributions or sudden spikes in TTL variance on specific subnets—driving the model’s anomaly score.

4 Visual Representation & Simulation Execution

For the hackathon presentation, the Oracle will power a fully offline Streamlit dashboard, demonstrating the end-to-end forward simulation.

- **The Probability Timeline:** A dynamic line chart plotting the scalar risk score across the K -step rollout, visually intersecting a predefined “Compromise Threshold.”
- **The Attack Topography:** An interactive graph representation highlighting the specific IP nodes and edges that the SHAP values have flagged as the epicenters of the predicted malicious activity.
- **Execution Flow:** The judge uploads a standard PCAP file. The pipeline extracts features, encodes the spatial graph, simulates the K -step future, and the Oracle updates the UI components dynamically based on the highest-confidence MITRE stage mapping.

The SOC “Glasshouse” Portal: UI/UX & Integration Architecture

4.1 The Integration Layer: Bridging the Engine to the Portal

The data emerging from our Transition Dynamics Engine (Part 3) is a continuous stream of simulated latent vectors ($\hat{Z}_{\text{future}}$). To render this in real-time without blocking the main event loop, we must decouple the inference engine from the presentation layer.

4.1.1 Architectural Pattern

We will pitch an asynchronous, event-driven architecture, even if the hackathon prototype runs locally.

- **The Backend (Inference Engine):** A local FastAPI instance wraps our PyTorch World Model. It ingests the PCAP/CSV files, runs the feature extraction, generates the K -step latent rollout, and passes the arrays through the CatBoost Oracle.
- **The Data Payload:** The backend serializes the complex outputs into a structured JSON response containing:

- Timestamp sequence (t through $t + K$).
- Overall Network Infiltration Probability for each timestep.
- Predicted MITRE ATT&CK stages.
- Graph-level data: Node/Edge lists with associated SHAP anomaly scores.
- Attention weights highlighting critical historical windows.

4.1.2 The “Hackathon to Production” Tech Stack

- **Hackathon Prototype:** Streamlit or Plotly Dash. These allow for rapid, Python-native UI development that meets the “offline-only” requirement perfectly, while still looking incredibly polished.
- **The Enterprise Vision (For the Pitch):** We will tell the judges that the production architecture envisions a React.js or Next.js frontend, utilizing WebSockets for ultra-low-latency streaming directly from edge nodes, rendered using WebGL-accelerated libraries like Deck.gl or Cytoscape.js.

4.2 Visualizing Temporal Analysis: The Three Core Components

To allow the client to “track the temporal analysis,” the dashboard must behave less like a static report and more like a video player for network state. We will divide the portal into three main visual zones.

4.2.1 Component A: The Causal Trajectory Timeline (The “When”)

This is the centerpiece of the temporal analysis.

- **Visual Design:** A large, interactive line chart spanning the bottom of the screen. The X-axis is time (past, present, and K -step simulated future). The Y-axis is the Infiltration Probability $[0, 1]$.
- **Mechanics:**
 - A solid blue line represents the historical and current network state up to time t .
 - A dotted red line extrapolates from t to $t + K$, representing the World Model’s forecast.
 - A horizontal red line marks the “Compromise Threshold” (e.g., 0.85). Where the dotted line crosses this threshold, a dynamic tag drops down labeling the impending MITRE ATT&CK stage (e.g., “PREDICTED: Lateral Movement in T+4 windows”).
- **Interactivity:** Users can click anywhere on the forecasted timeline to update the rest of the dashboard to reflect that specific simulated future moment.

4.2.2 Component B: The Dynamic Topological Threat Map (The “Where”)

A static list of IP addresses is tedious to read. We will visualize the network graph $\mathcal{G}_t$ dynamically.

- **Visual Design:** A force-directed graph network where nodes are endpoints (IPs/Ports) and edges are connections.
- **Temporal Shifting:** As the analyst scrubs the timeline slider forward into the simulated future, the nodes and edges animate.

- **Normal State:** Nodes pulse softly in green.
- **Forecasted Attack State:** If the Oracle predicts an anomaly on a specific subnet at $t + 3$, those specific nodes swell in size and transition to deep crimson. The edges (connections) that are predicted to carry the malicious payload thicken.
- **Implementation:** We can use Streamlit components integrated with `pyvis` or `Altair` to render interactive HTML network graphs directly in the web app.

4.2.3 Component C: The XAI “Root Cause” Sidebar (The “Why”)

Enterprise security teams will not block a critical subnet based on an AI’s hunch; they need mathematical proof.

- **Visual Design:** A dedicated right-hand sidebar that unpacks the black box.
- **Feature Attribution (SHAP Waterfall):** When a user clicks on a red (anomalous) node in the Threat Map, a SHAP waterfall chart instantly populates. It visually explains why the model thinks this node will be compromised:
 - +0.42 Anomaly Score: Unusually high Inter-Arrival Time (IAT) variance.
 - +0.35 Anomaly Score: 400% spike in TCP RST flags.
 - 0.10 Anomaly Score: Normal byte payload volume.
- **Temporal Attribution (Attention Heatmap):** A mini-heatmap showing the Transformer’s attention weights. It translates to layman terms: “This predicted Lateral Movement at $T + 5$ is highly correlated with a stealthy port sweep detected at $T - 15$.”

4.3 Implementation Snippet: The Streamlit Orchestrator

Here is a conceptual snippet of how we tie the engine into the interactive UI using Streamlit to create that “industry-grade” temporal scrubbing effect.

```

1 import streamlit as st
2 import pandas as pd
3 import plotly.express as px
4 from backend_engine import WorldModelPipeline, OracleXAI
5
6 # Set layout to wide for enterprise dashboard feel
7 st.set_page_config(layout="wide", page_title="N-WORLD SOC Portal")
8
9 # 1. Sidebar: Telemetry Ingestion
10 st.sidebar.header("Ingestion Engine")
11 uploaded_file = st.sidebar.file_uploader("Upload PCAP/CSV Telemetry",
12                                         type=['csv', 'pcap'])
13
14 if uploaded_file is not None:
15     # 2. Run the Offline Inference Pipeline
16     with st.spinner("Initializing World Model causal simulation..."):
17         pipeline = WorldModelPipeline()
18         historical_states, forecasted_states = pipeline.
19             process_and_simulate(uploaded_file, k_steps=10)
20
21         oracle = OracleXAI()
22         timeline_df = oracle.generate_risk_timeline(historical_states,
23                                                     forecasted_states)

```

```

22 # 3. Main UI: Causal Trajectory Timeline
23 st.title("Predictive Threat Landscape")
24
25 fig = px.line(timeline_df, x='Timestamp', y='
    Infiltration_Probability',
26               color='State_Type', # Distinguishes 'Observed' vs '
    Simulated'
27               line_dash='State_Type',
28               title="K-Step Forward Simulation Trajectory")
29 fig.add_hline(y=0.85, line_dash="dot", annotation_text="Critical
    Alert Threshold", annotation_position="bottom right")
30 st.plotly_chart(fig, use_container_width=True)
31
32 # 4. Interactive Temporal Scrubbing
33 st.subheader("Simulated State Explorer")
34 selected_time = st.slider("Scrub temporal forecast:",
35                           min_value=timeline_df['Timestamp'].min(),
36                           max_value=timeline_df['Timestamp'].max())
37
38 col1, col2 = st.columns([2, 1])
39
40 with col1:
41     # Dynamic Graph updating based on the slider's timestamp
42     st.markdown(f"**Topological Threat Map at {selected_time}**")
43     graph_html = oracle.render_network_graph_at_time(selected_time)
44     st.components.v1.html(graph_html, height=500)
45
46 with col2:
47     # XAI Root Cause Pane
48     st.markdown("**Explainability (XAI) Pane**")
49     st.info(f"**Predicted MITRE Stage:** {oracle.get_mitre_stage(
    selected_time)}")
50
51     st.markdown("### Top Driving Features")
52     shap_fig = oracle.plot_shap_waterfall(selected_time)
53     st.pyplot(shap_fig)

```

4.4 The Presentation Strategy

When presenting this, emphasize the human-in-the-loop philosophy. The AI does not replace the analyst; it gives them a time machine.

Use this specific narrative flow for the judges:

“Traditional dashboards show you what happened 5 minutes ago.”

“Our N-WORLD portal shows you what the network topology will look like 10 steps into the future.”

“By sliding the timeline forward, the analyst visually watches the attack pathway form before a single piece of malware is fully executed, backed by SHAP-driven root-cause explanations to ensure total trust in the prediction.”